

Labo 00 — De Linux-fundamenten die Docker als gekend beschouwt

Theorie — de kernel, processen, gebruikers, bestanden, de shell en drie netwerkbegrippen. Alles wat de volgende labo's gebruiken zonder het nog uit te leggen.

Doelstellingen

- De drie verdiepingen van een Linux-systeem situeren: kernel, system calls, userland.
- Een proces beschrijven: PID, ouder, omgeving, signalen, exitcode.
- Een regel van `ls -l` lezen: eigenaar, groep, permissies — en weten wat `root` meer mag dan jij.
- De shell als gereedschap gebruiken: omgevingsvariabelen, `PATH`, redirections, pipes.
- Elk begrip verbinden met wat het in de Docker-labo's zal worden.

1. Waarom een Linux-labo in een Docker-opleiding

Omdat een container **niets anders is dan Linux**. Wanneer je in labo 01 leest dat "een container een geïsoleerd proces is", heb je aan die zin alleen iets als *proces* voor jou een precies idee is: iets met een nummer, een ouder, een omgeving, een manier van sterven. Zo ook is het "rootless" van Podman onbegrijpelijk zonder UID's, een poort "die al bezet is" zonder het begrip poort, een volume zonder het begrip mount. Dit labo installeert die woordenschat, alleen die, en elke sectie kondigt aan in welk labo het begrip terugkomt.

→ GESCHIEDENIS

Unix ontstaat in 1969 bij Bell Labs en legt twee ideeën vast die nog altijd alles beheersen: *alles is een bestand* en *kleine programma's die je aan elkaar koppelt*. In 1991 schrijft een Finse student, Linus Torvalds, een vrije Unix-compatibele kernel: Linux. Ubuntu (2004) is er een **distributie** van: de Linux-kernel plus een gekozen en verpakte userland. En sinds 2019 levert Microsoft zijn eigen Linux-kernel mee in Windows: dat is WSL 2, jouw labomachine.

2. De kernel en de userland

Een Linux-systeem heeft twee verdiepingen. Onderaan de **kernel**: het enige programma dat de hardware aanraakt. Hij maakt processen aan, verdeelt CPU-tijd en geheugen, leest de schijven, stuurt netwerkpakketten. Bovenaan de **userland**: al de rest — `bash`, `ls`, `java`, jouw applicatie. Daartussen één enkele grens: de **system calls** (*syscalls*). Een programma leest nooit zelf een bestand; het vraagt `open` en dan `read` aan de kernel, die beslist of het mag.

Die grens verklaart twee dingen die je voortdurend zult zien. Eerst de portabiliteit: een Linux-binary werkt op elke distributie, want hij vraagt alleen system calls, overal identiek. Daarna de controle: omdat *alles* via de kernel passeert, volstaat het dat de kernel een beetje liegt ("jij bent proces 1", "dit is jouw `/`") om een proces te isoleren — precies wat een container in labo 01 zal doen.

→ WINDOWS / WSL

Een Linux-programma "spreekt" alleen met een Linux-kernel; Windows heeft zijn eigen, incompatibele kernel. **WSL 2** (*Windows Subsystem for Linux*) lost dat op door een echte Linux-kernel te draaien in een piepkleine VM die Windows beheert. Jouw Ubuntu 24.04 is een distributie *in* die VM: `uname -r` antwoordt er `...microsoft-standard-WSL2`, de handtekening van de door Microsoft gecompileerde kernel. De Windows-schijf is er zichtbaar onder `/mnt/c`.

3. Processen

Een **proces** is een programma in uitvoering: code, geheugen en een identiteit. De kernel geeft het een uniek **PID** (*process ID*) en onthoudt zijn ouder, het **PPID**. Elk proces wordt uit een ander geboren — als je `ls` typt, dupliceert je shell zichzelf (`fork`) en vervangt de kloon zich door `ls` (`exec`). Bij het opstarten lanceert de kernel een eerste proces, **PID 1** (`systemd` op Ubuntu), voorouder van alle andere; sterft een ouder vóór zijn kind, dan wordt de wees geadopteerd door PID 1. Onthoud dat nummer: in een container is *jouw applicatie* PID 1, met onverwachte verantwoordelijkheden (labo 03).

Een proces eindigt altijd met een **exitcode**: `0` betekent "succes", al de rest is een mislukking. De shell bewaart ze in `$?`. Enkele conventionele waarden: `1` algemene fout, `2` verkeerd gebruik, `126` bestand niet uitvoerbaar, `127` commando niet gevonden, `128 + n` gedood door signaal *n*.

Want een proces "sluit" je niet af: je stuurt het een **signaal**, een genummerde melding van de kernel. De drie die je moet kennen:

Signaal	Nummer	Betekenis	Kan het proces het negeren?
<code>SIGTERM</code>	15	"Beëindig jezelf netjes"	Ja — het mag eerst opslaan, afsluiten, opruimen
<code>SIGKILL</code>	9	Onmiddellijke dood, door de kernel	Nee — en niets wordt opgeruimd
<code>SIGINT</code>	2	Toetsenbordonderbreking (<code>Ctrl+C</code>)	Ja

ONTHOUDEN

`kill` betekent niet "doden" maar "een signaal sturen"; standaard stuurt het het beleefde `SIGTERM`. Een proces gedood door `SIGKILL` eindigt met code `137` (`128 + 9`). Dat getal zul je je hele Docker-leven terugzien: het is de handtekening van een hardhandig gestopte container — vaak wegens geheugengebrek.

De kernel toont de toestand van elk proces in `/proc`, een nepmap: `/proc/1234/` beschrijft proces 1234 (zijn commando, zijn omgeving, zijn limieten), ter plekke gefabriceerd, zonder één byte schijfruimte. `ps` doet niets anders dan dat lezen.

4. Gebruikers, groepen, permissies

Elk proces draait *a/s* iemand: een **gebruiker**, geïdentificeerd door een nummer, de **UID**, en **groepen** (GID). De kernel kent alleen nummers; de namen (`kevin`, `postgres`) komen uit het bestand `/etc/passwd`. Je eerste Ubuntu-gebruiker heeft UID **1000**. De gebruiker `root`, UID **0**, is speciaal: de kernel weigert hem niets. Het commando `sudo` voert een commando uit *a/s* root, en logt wie erom vroeg.

Elk bestand heeft een eigenaar, een groep en negen permissiebits, leesbaar in `ls -l`:

```
-rw-r----- 1 root shadow 1234 ... /etc/shadow
```

└─┬─┬─┐ ┌───┐ eigenaar root, groep shadow
| | └─┘ anderen: niets
| └─┘ groep shadow: lezen
└─┘ root: lezen + schrijven

`r` lezen, `w` schrijven, `x` uitvoeren (voor een map: binnengaan). `chmod` wijzigt die bits, `chown` de eigenaar. Een detail waar iedereen invliegt: een script moet **uitvoerbaar** zijn (`chmod +x`) om met `./script.sh` gestart te worden — anders antwoordt de shell `Permission denied`, code 126.

→ BEVEILIGING

De gouden regel: je werkt nooit als root, je verhoogt je rechten punctueel met `sudo`. Dat is de Linux-versie van het principe van minimale rechten, en het centrale argument van Podman **rootless**: jouw containers draaien onder UID 1000, niet onder UID 0, en een gecompromitteerde applicatie heeft alleen jouw rechten (labo 01).

VALKUIL

De kernel vergelijkt **nummers**, geen namen. Een bestand aangemaakt door UID 1000 in een container behoort overal toe aan UID 1000, ook al verandert de getoonde naam van systeem tot systeem. Die evidentie wordt in labo 06 de klassieke hoofdbreker van volumes.

5. Bestanden, boomstructuur, mounts

Onder Unix *is alles een bestand*: documenten, maar ook schijven (`/dev/sdc`), de toestand van de kernel (`/proc`), sockets. Er zijn geen stations `C:` of `D:`: één enkele boom, vertrekkend van de root `/`, waar elke schijf of elk bestandssysteem wordt **gemount** — vastgehaakt aan een map. `findmnt /` vertelt je welke schijf de root levert; op WSL is `/mnt/c` de mount van de Windows-schijf. Mounten, unmounten, bestandssystemen stapelen: dat is exact de mechaniek van Docker-images en -volumes (labo's 02 en 06).

De standaardmappen om te herkennen: `/etc` (configuratie), `/home` (jouw bestanden), `/usr/bin` (de programma's), `/var` (levende data: logs, databanken), `/tmp` (tijdelijk), `/proc` en `/sys` (vensters op de kernel).

6. De shell: de omgeving, het PATH, het loodgieterswerk

De **shell** (`bash`) is een proces als een ander, met als taak de andere te lanceren. Drie van zijn mechanismen zijn pure "Docker-kennis".

De omgevingsvariabelen. Elk proces wordt geboren met een woordenboek sleutel=waarde geërfd van zijn ouder: `HOME`, `PATH`, `LANG` ... Een shellvariabele (`MSG=hallo`) blijft lokaal; ze komt pas in de omgeving van de kinderen na `export MSG`. Dit is HET configuratiekanaal van containers: in labo 08 leest je Spring Boot-applicatie haar databankwachtwoord uit een variabele, nooit uit een bestand in de image.

→ **JAVA**

Een JVM is een gewoon proces: `java -jar app.jar` heeft een PID, een UID, variabelen. Spring Boot leest de omgeving bij het opstarten: `SERVER_PORT=9090` volstaat om zijn poort te veranderen, zonder de JAR aan te raken. `System.getenv("HOME")` in Java is het lezen van datzelfde geërfde woordenboek.

Het PATH. Als je `ls` typt, zoekt de shell een uitvoerbaar bestand met de naam `ls` in de lijst mappen van de variabele `PATH`, in volgorde. `which ls` toont wat hij vond; `command not found`

(code 127) betekent "in geen van die mappen". Daarom start je een script uit de huidige map met `./script.sh`: "hier" zit niet in het `PATH`, uit voorzichtigheid.

Redirections en pipes. Een proces heeft drie stromen: de invoer (0, *stdin*), de uitvoer (1, *stdout*) en de fouten (2, *stderr*). De shell sluit ze aan waar je wilt: `> bestand` leidt de uitvoer om, `2>` de fouten, `2>&1` voegt beide samen, en `commando1 | commando2` sluit de uitvoer van het ene aan op de invoer van het andere. Je zult die buizen in alle labo's assembleren (`podman ps | grep ...`), en de logs van een container zijn niets anders dan zijn stromen 1 en 2, opgevangen (labo 03).

7. Het netwerk in drie begrippen

Je hebt drie ideeën nodig om labo 07 te halen. **De interface:** het netwerkstopcontact van een machine, met een IP-adres; `lo`, de *loopback*-interface, draagt het adres `127.0.0.1`, alias `localhost` — de machine die tegen zichzelf praat. **De poort:** een nummer van 1 tot 65535 dat de diensten van eenzelfde adres onderscheidt; één enkel proces luistert op een gegeven poort, `ss -tlnp` toont wie waar luistert. **Het privilege:** poorten onder 1024 zijn voorbehouden aan root — de reden waarom je testserver op 8080 zal luisteren en niet op 80, en waarom Podman rootless `-p 80:80` zal weigeren (labo 07).

→ NETWORK

`curl` is het Zwitsers zakmes: het doet een HTTP-verzoek en toont het rauwe antwoord. `curl -i http://localhost:8080/` toont de code (`200 OK`, `404 ...`), de headers, de body. Hét gereedschap nummer één om een gecontaineriseerde API te testen zonder browser.

→ WINDOWS / WSL

WSL 2 stuurt `localhost` automatisch door: een server die op poort 8080 luistert in Ubuntu is bereikbaar vanuit een **Windows**-browser op `http://localhost:8080`. Handig, maar onthoud dat die doorschakeling een gunst van WSL is, geen eigenschap van Linux.

8. In het bedrijf

Het hele container-ecosysteem is de industrialisering van deze begrippen. Een Spring Boot-productieserver, dat is: een `java`-proces (PID) gestart door een applicatiegebruiker zonder rechten (UID), geconfigureerd via omgevingsvariabelen, dat zijn logs naar *stdout* schrijft, luistert op poort 8080, en bij deployments wordt gestopt met `SIGTERM`. De beheerder die een incident onderzoekt, rijgt `ps`, `ss` en `curl` aan elkaar, leest `$?`, en doorzoekt de logs met `grep`. Docker vervangt daar niets van: het verpakt het.

PODMAN

Podman trekt die logica helemaal door: geen dienst op de achtergrond, alleen *jouw* gebruiker (UID 1000) die processen start. Heel dit labo — UID's, signalen, `/proc`, niet-geprivilegieerde poorten — is de exacte beschrijving van wat Podman zonder `sudo` mag doen. Docker steunt daarentegen op een dienst die als root draait; dat verschil vult labo 01.

Onthouden

- De **kernel** controleert alles; programma's doen alleen **system calls**. Een proces isoleren is de kernel doen liegen — het stichtende idee van de container.
- Een **proces** heeft een PID, een ouder, een geërfde omgeving, en eindigt met een **exitcode**: `0` = succes, `137` = gedood door SIGKILL.

- `SIGTERM` vraagt beleefd, `SIGKILL` executeert zonder beroep. Een goed opgevoede dienst stopt op `SIGTERM`.
- De kernel redeneert in numerieke **UID/GID**; `root` = UID 0 = alle rechten; `sudo` verhoogt punctueel.
- Eén enkele bestandsboom; schijven en virtuele systemen worden erin **gemount**; `/proc` is het venster op de kernel.
- **Omgevingsvariabelen** gaan van ouder naar kind; het `PATH` beslist welke commando's bestaan.
- Een dienst = een adres + een **poort**; `localhost` = de machine zelf; poorten < 1024 alleen voor root.

Woordenschat

kernel : het programma dat hardware en processen controleert. — **userland** : alles wat boven de kernel draait. — **system call** : verzoek van een programma aan de kernel (`open` , `fork` ...). — **proces** : programma in uitvoering, geïdentificeerd door een **PID**. — **PID 1** : eerste proces, voorouder en voogd van alle andere. — **signaal** : melding gestuurd naar een proces (`SIGTERM` , `SIGKILL`). — **exitcode** : geheel getal bij de dood van een proces, `0` = succes, in `$?` . — **UID / GID** : gebruikers- en groepsnummers, het enige wat de kernel begrijpt. — **root** : UID 0, geen enkele controle is van toepassing. — **mount** : vasthaken van een bestandssysteem aan een map van de boom. — **/proc** : virtuele boomstructuur die de toestand van kernel en processen toont. — **omgevingsvariabele** : paar sleutel=waarde geërfd door kindprocessen. — **PATH** : lijst mappen waar de shell commando's zoekt. — **stdin / stdout / stderr** : de drie standaardstromen (0, 1, 2). — **pipe** : aansluiting van de uitvoer van een proces op de invoer van een ander. — **poort** : nummer dat een dienst op een IP-adres identificeert. — **localhost** : `127.0.0.1` , het adres van de machine zelf.